Name: Domaguio, John Joseph C. Course/Section: CPE 311-CPE22S3 Instructor: NEAL BARTON JAMES MATIRA

Question 1

Your goal in this data is to implement a difference function, which subtracts one list from another and returns the result.

It should remove all values from list a, which are present in list b keeping their order.

```
array_diff([1,2],[1]) == [2]
```

If a value is present in b, all of its occurrences must be removed from the other:

```
array_diff([1,2,2,2,3],[2]) == [1,3]
```

```
def array_diff(a, b):

    ## Code here
    result = []
    for item in a:
        if item not in b:
            result.append(item)


    return result

# @title Test code should run and not return error
def test_1():
    sample = ([1,2], [1])
    assert(array_diff(sample[0],sample[1])) == [2]
def test_2():
    sample = ([1,2,2], [1])
    assert(array_diff(sample[0],sample[1])) == [2,2]
def test_3():
    sample = ([1,2,2], [2])
    assert(array_diff(sample[0],sample[1])) == [1]
def test_4():
    sample = ([1,2,2], [])
    assert(array_diff(sample[0],sample[1])) == [1,2,2]
def test_5():
    sample = ([], [1,2])
    assert(array_diff(sample[0],sample[1])) == []
def test_6():
    sample = ([1,2,3], [1, 2])
    assert(array_diff(sample[0],sample[1])) == [3]

test_1()
```

```
test_2()
test_3()
test_4()
test_5()
test_6()
```

Analysis Here What i did is to write the funcion that will compare the elements of the first list that is also found in the second list while keeping the order the same.

Question 2

Your task is to make a function that can take any non-negative integer as an argument and return it with its digits in descending order. Essentially, rearrange the digits to create the highest possible number.

Examples: Input: 42145 Output: 54421

Input: 145263 Output: 654321

Input: 123456789 Output: 987654321

```
def ips_between(num):

    ## Answer Here
    digits = list(str(num))
    digits.sort(reverse=True)
    result = int("".join(digits))


    return result

# @title Test code should run and not return error

def test_1():
    assert(ips_between(0)== 0)
def test_2():
    assert(ips_between(15)== 51)
def test_3():
    assert(ips_between(123456789)== 987654321)
test_1()
test_2()
test_3()
```

Analysis Here This problem reaarrange the digits of a number into descending order to form the biggest possible number.

Question 3

Complete the square sum function so that it squares each number passed into it and then sums the results together. For example, for [1, 2, 2] it should return 9 because 12+22+22=9.

```
def square_sum(numbers):
    #your code here
    squaredSum = 0
    for number in numbers:
        squaredSum += number ** 2

    return squaredSum

# @title Test code should run and not return error

def test_1():
    sample = [1,2]
    assert(square_sum(sample)) == 5
def test_2():
    sample = [0,3,4,5]
    assert(square_sum(sample)) == 50
def test_3():
    sample = []
    assert(square_sum(sample)) == 0
def test_4():
    sample = [-1,-2]
    assert(square_sum(sample)) == 5
def test_5():
    sample = [-1,0,1]
    assert(square_sum(sample)) == 2

test_1()
test_2()
test_3()
test_4()
test_5()
```

Analysis Here This problem reverses all words that have more letters in a sentence

Question 4

Write a function that takes in a string of one or more words, and returns the same string, but with all words that have five or more letters reversed (Just like the name of this Kata). Strings passed in will consist of only letters and spaces. Spaces will be included only when more than one word is present.

Examples:

```
"Hey fellow warriors"  --> "Hey wollef sroirraw"
"This is a test        --> "This is a test"
"This is another test" --> "This is rehtona test"

def spin_words(sentence):

    # Answer here
    words = sentence.split()
    new_words = []

    for word in words:
        if len(word) >= 5:
            new_words.append(word[::-1])
        else:
            new_words.append(word)

    result = " ".join(new_words)


    return result

# @title Test code should run and not return error

def test_1():
    sample = "Welcome"
    assert(spin_words(sample)) == "emocleW"
def test_2():
    sample = "to"
    assert(spin_words(sample)) == "to"
def test_3():
    sample = "CodeWars"
    assert(spin_words(sample)) == "sraWedoC"
def test_4():
    sample = "Hey fellow warriors"
    assert(spin_words(sample)) == "Hey wollef sroirraw"
def test_5():
    sample = "This sentence is a sentence"
    assert(spin_words(sample)) == "This ecnetnes is a ecnetnes"

test_1()
test_2()
test_3()
test_4()
test_5()

---------------------------------------------------------------------------
-----
ModuleNotFoundError                       Traceback (most recent call
last)
/tmp/ipython-input-636360411.py in <cell line: 0>()
      1 # @title
```

```
----> 2 from main import spin_words
      3 def test_1():
      4     sample = "Welcome"
      5     assert(spin_words(sample)) == "emocleW"

ModuleNotFoundError: No module named 'main'

--------------------------------------------------------------------------
-----
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
--------------------------------------------------------------------------
-----
```

Analysis Here

Question 5

DESCRIPTION: Well met with Fibonacci bigger brother, AKA Tribonacci.

As the name may already reveal, it works basically like a Fibonacci, but summing the last 3 (instead of 2) numbers of the sequence to generate the next. And, worse part of it, regrettably I won't get to hear non-native Italian speakers trying to pronounce it :(

So, if we are to start our Tribonacci sequence with [1, 1, 1] as a starting input (AKA signature), we have this sequence:

```
[1, 1 ,1, 3, 5, 9, 17, 31, ...]
```

But what if we started with [0, 0, 1] as a signature? As starting with [0, 1] instead of [1, 1] basically shifts the common Fibonacci sequence by once place, you may be tempted to think that we would get the same sequence shifted by 2 places, but that is not the case and we would get:

```
[0, 0, 1, 1, 2, 4, 7, 13, 24, ...]
```

Well, you may have guessed it by now, but to be clear: you need to create a fibonacci function that given a signature array/list, returns the first n elements - signature included of the so seeded sequence.

Signature will always contain 3 numbers; n will always be a non-negative number; if n == 0, then return an empty array (except in C return NULL) and be ready for anything else which is not clearly specified ;)

If you enjoyed this kata more advanced and generalized version of it can be found in the Xbonacci kata

[Personal thanks to Professor Jim Fowler on Coursera for his awesome classes that I really recommend to any math enthusiast and for showing me this mathematical curiosity too with his usual contagious passion :)]

```
def tribonacci(signature, n):
  ## Answer Here
   if n == 0:
        return []
   if n <= 3:
    return signature[:n]

   result = signature[:]
   while len(result) < n:
       result.append(result[-1] + result[-2] + result[-3])

   return result

# @title Test code should run and not return error

def test_1():
    sample = ([1, 1, 1], 10)
    assert(tribonacci(sample[0],sample[1])) == [1, 1, 1, 3, 5, 9, 17,
31, 57, 105]
def test_2():
    sample = ([0, 0, 1], 10)
    assert(tribonacci(sample[0],sample[1])) == [0, 0, 1, 1, 2, 4, 7,
13, 24, 44]
def test_3():
    sample = ([0, 1, 1], 10)
    assert(tribonacci(sample[0],sample[1])) == [0, 1, 1, 2, 4, 7, 13,
24, 44, 81]
def test_4():
    sample = ([1, 0, 0], 10)
    assert(tribonacci(sample[0],sample[1])) == [1, 0, 0, 1, 1, 2, 4,
7, 13, 24]
def test_5():
    sample = ([0, 0, 0], 10)
    assert(tribonacci(sample[0],sample[1])) == [0, 0, 0, 0, 0, 0, 0,
0, 0, 0]
def test_6():
    sample = ([1, 2, 3], 10)
    assert(tribonacci(sample[0],sample[1])) == [1, 2, 3, 6, 11, 20,
37, 68, 125, 230]
def test_7():
    sample = ([3, 2, 1], 10)
    assert(tribonacci(sample[0],sample[1])) == [3, 2, 1, 6, 9, 16, 31,
56, 103, 190]
```

```
def test_8():
    sample = ([1, 1, 1], 1)
    assert(tribonacci(sample[0],sample[1])) == [1]
def test_9():
    sample = ([300, 200, 100], 0)
    assert(tribonacci(sample[0],sample[1])) == []
def test_10():
    sample = ([0, 0, 1], 10)
    assert(tribonacci(sample[0],sample[1])) == [0, 0, 1, 1, 2, 4, 7,
13, 24, 44]
def test_11():
    sample = ([0.5, 0.5, 0.5], 30)
    assert(tribonacci(sample[0],sample[1])) == [0.5, 0.5, 0.5, 1.5,
2.5, 4.5, 8.5, 15.5, 28.5, 52.5, 96.5, 177.5, 326.5, 600.5, 1104.5,
2031.5, 3736.5, 6872.5, 12640.5, 23249.5, 42762.5, 78652.5, 144664.5,
266079.5, 489396.5, 900140.5, 1655616.5, 3045153.5, 5600910.5,
10301680.5]

test_1()
test_2()
test_3()
test_4()
test_5()
test_6()
test_7()
test_8()
test_9()
test_10()
test_11()
```

Analysis Here This question generate a sequence where each number is the sum of the previous three numbers.

Conclusion In this activity, I learned how to solve different problems using simple Python codes and basic logic. Each problem helped me understand how loops, conditions, and lists work together in programming. Overall, these exercises improved my problem solving skills and made coding more easier to understand.